AMAZON CLONE

Documentación Oficial del Proyecto

Versión del documento	1.1.0
Fecha de emisión	2026-10-11
Estado	En desarrollo
Autor	Dylan Dominguez Perez
Contacto	github.com/DylanDominguez
Audiencia	Equipo de desarrollo
Commit documentado	`49fa7c2`

Documento destinado al equipo de desarrollo. La fuente de verdad es **docs/DOCUMENTACION.md**; este PDF es un entregable generado automáticamente.

Índice

1. Información del documento
    1.1 Historial de versiones del documento
2. Resumen ejecutivo
    2.1 Propósito
    2.2 Objetivos
    2.3 Alcance
3. Stack tecnológico
    3.1 Herramientas de documentación
4. Arquitectura del proyecto
    4.1 Estructura actual del repositorio
    4.2 Estructura de la aplicación (amazon-clone/src/)
    4.3 Modelo arquitectónico
5. Instalación y arranque
    5.1 Generar la documentación
    5.2 Ejecutar la aplicación (amazon-clone/)
6. Flujo de trabajo del equipo
    6.1 Ramas
    6.2 Ciclo de una funcionalidad
    6.3 Convenciones de commit
7. Guía de desarrollo y convenciones
    7.1 Convenciones de código
    7.2 Estructura de un componente
    7.3 Reglas del equipo
8. Inventario de componentes
9. Rutas y navegación
10. Estado global y datos
11. Sistema de diseño
12. Roadmap / Plan de trabajo
13. Historial de cambios (Changelog)

v1.1.0 — 2026-10-11 — 49fa7c2

v1.0.0 — 2026-10-11 — 9bb4aa8

14. Proceso de actualización de esta documentación

14.1 Cuándo actualizarlo

14.2 Procedimiento

14.3 Fuente de verdad

> **Cómo usar este documento.** Esta es la fuente oficial de información del proyecto. Todo lo que el equipo necesita saber (arquitectura, convenciones, estado del código y decisiones tomadas) se documenta aquí. El PDF se genera a partir de este archivo Markdown con `npm --prefix docs run generar`. Las secciones marcadas como **Pendiente** corresponden a funcionalidades aún no implementadas y se completarán en futuras versiones de este documento.

---

1. Información del documento

Campo	Valor
Proyecto	Amazon Clone with React
Versión del documento	1.1.0
Fecha de emisión	2026-10-11
Estado	En desarrollo (fase inicial)
Autor	Dylan Dominguez Perez
Audiencia	Equipo de desarrollo
Commit documentado	`49fa7c2`
Repositorio	`Amazon-Clone-with-React` (rama `main`)

1.1 Historial de versiones del documento

Versión	Fecha	Commit	Responsable	Descripción
1.1.0	2026-10-11	`49fa7c2`	Dylan Dominguez Perez	Inicialización de la app React en `amazon-clone/` con Create React App, configuración de Firebase, stack confirmado (React 19, CRA), estructura real documentada, comandos de arranque actualizados, inventario de componentes inicial.
1.0.0	2026-10-11	`9bb4aa8`	Dylan Dominguez Perez	Creación de la documentación oficial: estructura base, stack declarado, guías iniciales y proceso de mantenimiento.

---

2. Resumen ejecutivo

Amazon Clone with React es una aplicación web frontend que replica la experiencia principal de compra de Amazon. El proyecto está concebido como una plataforma de comercio electrónico con catálogo de productos, búsqueda, carrito de compras y flujo de compra.

El objetivo técnico del proyecto es construir una interfaz basada en **componentes reutilizables**, con código mantenible y una arquitectura de frontend moderna.

2.1 Propósito

- Replicar la interfaz y los flujos principales de la tienda Amazon.
- Aplicar y consolidar prácticas modernas de desarrollo frontend con React.
- Mantener un código organizado, modular y fácil de mantener a lo largo del tiempo.

2.2 Objetivos

1. Interfaz responsiva y funcional en distintos tamaños de pantalla.
2. Arquitectura de componentes reutilizables y con responsabilidades claras.
3. Gestión predecible del estado de la aplicación (carrito, sesión, filtros).
4. Código fuente acompañado de esta documentación oficial, actualizada en cada avance.

2.3 Alcance

Pendiente de definición. El detalle de funcionalidades (features) del clon se definirá con el equipo y se documentará en la sección 12 (Roadmap). El estado actual incluye la inicialización del proyecto React con configuración de Firebase.

---

3. Stack tecnológico

Tecnología	Rol	Justificación	Estado
⚛️ React 19.3.0	Librería de UI	Arquitectura por componentes, ecosistema maduro, concurrent features	✅ Instalada
🟨 JavaScript (ES6+)	Lógica de la aplicación	Lenguaje estándar del ecosistema React	✅ En uso
🎨 CSS3	Estilos y layout	Estilos responsivos sin dependencias adicionales	✅ En uso
🔧 Create React App (react-scripts 5.0.1)	Build tool y dev server	Configuración cero, ampliamente adoptado, buen soporte	✅ Instalada
🔥 Firebase (config)	Backend as a Service	Autenticación, base de datos, hosting, analytics	⚙️ Configurada (SDK pendiente)
📦 npm	Gestión de dependencias	Empaquetado oficial de Node.js	✅ En uso
🟩 Node.js	Runtime de herramientas	Ejecución de scripts de build y documentación	≥ 24 (entorno actual)
🧪 Testing Library / Jest	Testing	Testing unitario y de integración incluido en CRA	✅ Incluido

> **Nota:** El stack original preveía Vite; la decisión final fue Create React App (react-scripts). Firebase SDK (`firebase` package) aún no está en `dependencies` — solo existe el archivo de configuración `src/firebase.js` . Se añadirá cuando se implemente autenticación o base de datos.

3.1 Herramientas de documentación

Herramienta	Rol
Markdown (`DOCUMENTACION.md`)	Fuente de verdad del documento
`marked`	Conversión de Markdown a HTML
Puppeteer	Renderizado e impresión a PDF (A4)
`docs/generar-pdf.mjs`	Script de generación del PDF

4. Arquitectura del proyecto

4.1 Estructura actual del repositorio

```
Amazon-Clone-with-React/
├── .git/                          # Control de versiones
├── README.md                      # Presentación del proyecto (raíz)
├── amazon-clone/                  # Aplicación React (Create React App)
│   ├── .gitignore                 # Ignora node_modules, build, .env, etc.
│   ├── package.json               # Dependencias y scripts de la app
│   ├── package-lock.json          # Lockfile
│   ├── public/                    # Archivos estáticos servidos tal cual
│   │   ├── favicon.ico
│   │   ├── index.html             # HTML base (punto de entrada)
│   │   ├── logo192.png            # Icono PWA 192x192
│   │   ├── logo512.png            # Icono PWA 512x512
│   │   ├── manifest.json          # PWA manifest
│   │   └── robots.txt
│   └── src/                       # Código fuente de la aplicación
│       ├── index.js               # Punto de entrada (React 18+ createRoot)
│       ├── App.js                 # Componente raíz
│       ├── App.css                # Estilos del componente App
│       ├── index.css              # Estilos globales
│       ├── firebase.js            # Configuración de Firebase
│       ├── logo.svg               # Logo React (boilerplate)
│       ├── reportWebVitals.js     # Medición de performance (web-vitals)
│       ├── setupTests.js          # Configuración de Testing Library
│       └── App.test.js            # Test de ejemplo
├── docs/                          # Sistema de documentación oficial
│   ├── .gitignore
│   ├── package.json
│   ├── DOCUMENTACION.md           # Fuente de la documentación (este archivo)
│   ├── plantilla.css
│   ├── generar-pdf.mjs
│   └── pdf/                       # PDFs generados (entregables)
```

4.2 Estructura de la aplicación (`amazon-clone/src/`)

Archivo / Carpeta	Descripción
`index.js`	Punto de entrada: `ReactDOM.createRoot` + `React.StrictMode` + `reportWebVitals`
`App.js`	Componente raíz (boilerplate CRA con logo y enlace a React)
`App.css` / `index.css`	Hojas de estilo del componente y globales
`firebase.js`	Objeto `firebaseConfig` con credenciales del proyecto `project-baf04`
`setupTests.js`	`@testing-library/jest-dom` para matchers extendidos en tests
`reportWebVitals.js`	Opcional: envío de métricas de performance a Analytics
`public/index.html`	HTML base con `%PUBLIC_URL%` para assets, meta tags PWA

4.3 Modelo arquitectónico

- **Component-based:** la UI se compone de componentes pequeños, reutilizables y con una única responsabilidad.
- **Flujo de datos unidireccional:** el estado desciende mediante props; los eventos ascienden mediante callbacks.
- **Separación de capas (objetivo a medida que crezca):**
  - `src/components/` — Componentes reutilizables de UI
  - `src/pages/` — Vistas asociadas a rutas
  - `src/hooks/` — Hooks personalizados
  - `src/context/` — Estado global (React Context)
  - `src/services/` — Acceso a datos / API / Firebase
  - `src/assets/` — Imágenes y recursos estáticos
  - `src/styles/` — Hojas de estilo globales / variables CSS

---

5. Instalación y arranque

5.1 Generar la documentación

```
# 1. Instalar dependencias (solo la primera vez)
npm --prefix docs install

# 2. Generar el PDF
npm --prefix docs run generar
```

El PDF se escribe en `docs/pdf/Documentacion-Oficial-v<versión>.pdf` .

5.2 Ejecutar la aplicación (`amazon-clone/`)

```
# 1. Instalar dependencias
cd amazon-clone
npm install

# 2. Servidor de desarrollo (puerto 3000)
npm start

# 3. Build de producción (carpeta build/)
npm run build

# 4. Tests en modo watch
npm test
```

> **Puerto por defecto:** 3000 (CRA). Si está ocupado, CRA pedirá usar otro.

---

6. Flujo de trabajo del equipo

6.1 Ramas

Rama	Propósito
`main`	Estado estable y aprobado del proyecto
`feature/<nombre>`	Desarrollo de una funcionalidad nueva
`fix/<nombre>`	Corrección de un bug

6.2 Ciclo de una funcionalidad

1. Crear la rama desde `main` .
2. Desarrollar y probar localmente (`npm start` / `npm test`).
3. Actualizar la documentación si corresponde (ver sección 14).
4. Merge a `main` con mensaje de commit convencional.
5. Regenerar el PDF oficial si hubo cambios relevantes.

6.3 Convenciones de commit

Formato: `tipo: descripción corta` — tipos: `feat` , `fix` , `docs` , `style` , `refactor` , `chore` .

Ejemplo: `feat: agregar tarjeta de producto en el home` .

---

7. Guía de desarrollo y convenciones

7.1 Convenciones de código

Área	Convención
Archivos de componentes	`PascalCase.js` / `PascalCase.jsx` (ej. `ProductCard.js`)
Componentes	Funcionales + hooks
Variables y funciones	`camelCase`
Componentes y tipos	`PascalCase`
Constantes globales	`SCREAMING_SNAKE_CASE`
Estilos	Archivo CSS por componente (`Componente.css`) o carpeta `styles/`
Props	Validar tipos cuando la complejidad lo justifique (PropTypes o TypeScript futuro)
Imports	Relativos para código interno; absolutos (`@/...`) si se configura alias

7.2 Estructura de un componente

```
function Componente({ prop1, prop2 }) {
  return (
    <div className="componente">
      {/* markup */}
    </div>
  );
}
export default Componente;
```

7.3 Reglas del equipo

- Un componente = un archivo = una responsabilidad.
- No duplicar lógica: extraer a hook o utilidad compartida.
- Toda funcionalidad nueva debe quedar reflejada en este documento.
- Los comentarios explican el *por qué*, no el *qué*.
- Mantener `package-lock.json` en control de versiones.

---

8. Inventario de componentes

Estado: inicial — componentes base de CRA.

Componente	Ubicación	Props	Responsabilidad
`App`	`src/App.js`	—	Componente raíz (boilerplate, será reemplazado)
`ReportWebVitals`	`src/reportWebVitals.js`	`onPerfEntry`	Medición opcional de performance (CLS, FID, LCP, etc.)

> **Nota:** `App.js` actual es el boilerplate de CRA. Se reemplazará por la estructura real del clon (header, home, productos, carrito, etc.) en próximas versiones.

---

9. Rutas y navegación

Estado: pendiente — CRA no incluye router por defecto.

Ruta	Página	Descripción
`/`	`App`	Página actual (boilerplate)

> **Decisión pendiente:** se usará `react-router-dom` cuando se implementen las vistas (home, producto, carrito, checkout, cuenta).

---

10. Estado global y datos

Estado: pendiente.

- **Firebase Config:** `src/firebase.js` expone `firebaseConfig` con credenciales del proyecto `project-baf04` (Auth, Firestore, Storage, Analytics).
- **SDK de Firebase:** *no instalado aún* en `dependencies`. Se añadirá (`npm install firebase`) al implementar autenticación o base de datos.
- **Estrategia de estado global:** por definir (React Context + useReducer, Zustand, Redux Toolkit, etc.). Se documentará en v1.2.0+.

---

11. Sistema de diseño

Estado: pendiente. Paleta de colores, tipografía, espaciados y patrones visuales replicados de Amazon se documentarán en esta sección.

> **Referencia visual:** Amazon usa principalmente `#131921` (navbar), `#232f3e` (footer), `#ff9900` (botones primarios / acentos), tipografía `Amazon Ember` / fallback sans-serif.

---

12. Roadmap / Plan de trabajo

Pendiente de definición con el equipo. Esta sección contendrá las fases del proyecto y el estado de cada una.

Fase	Entregable	Estado
0	Documentación oficial del proyecto	✅ Completada (v1.0.0)
1	Inicialización del proyecto (CRA + React 19 + Firebase config)	✅ Completada (v1.1.0)
2	Instalar Firebase SDK + Auth (login/registro)	⬜ Pendiente
3	Estructura de rutas + vistas base (Home, Producto, Carrito, Checkout)	⬜ Pendiente
4	Catálogo de productos + búsqueda + filtros	⬜ Pendiente
5	Carrito persistente + flujo de compra	⬜ Pendiente
6	Historial de pedidos + cuenta de usuario	⬜ Pendiente
7	Testing, accesibilidad, performance, despliegue	⬜ Pendiente

---

13. Historial de cambios (Changelog)

v1.1.0 — 2026-10-11 — `49fa7c2`

- Inicialización de la aplicación React en `amazon-clone/` con **Create React App** (react-scripts 5.0.1).
- **React 19.3.0** confirmado como versión de runtime.
- Configuración de **Firebase** en `src/firebase.js` (proyecto `project-baf04`, Auth + Firestore + Storage + Analytics).
- Estructura real del proyecto documentada (sección 4.1 y 4.2 actualizadas).
- Stack tecnológico actualizado: CRA en lugar de Vite, Firebase configurado, Testing Library incluido.
- Guía de instalación y arranque con comandos reales (`npm start` , `npm run build` , `npm test`).
- Inventario de componentes inicial (`App` , `ReportWebVitals`).
- Roadmap actualizado con fases concretas.
- Documentación de la decisión de build tool (CRA vs Vite original).

v1.0.0 — 2026-10-11 — `9bb4aa8`

- Creación del sistema de documentación oficial (`docs/`).
- Estructura completa del documento: resumen ejecutivo, stack, arquitectura, guías de instalación y desarrollo, convenciones, flujos de trabajo Git.
- Secciones de inventario (componentes, rutas, estado, diseño) preparadas para poblar según avance el código.
- Script generador de PDF (`generar-pdf.mjs`) con portada, índice y paginación.
- Definición del proceso oficial de actualización de la documentación (sección 14).

---

14. Proceso de actualización de esta documentación

Este documento es **oficial** y se mantiene actualizado en cada avance relevante del proyecto.

14.1 Cuándo actualizarlo

- Al agregar una funcionalidad nueva.
- Al modificar la arquitectura, estructura de carpetas o stack.
- Al cambiar convenciones del equipo.
- Al definir o modificar el roadmap.

14.2 Procedimiento

1. **Analizar el cambio:** revisar el código fuente y ejecutar `git log` desde el `commit_base` indicado en la portada.
2. **Actualizar las secciones afectadas** de `docs/DOCUMENTACION.md`.
3. **Incrementar la versión** según semver del documento:
   - *mayor:* reestructuración general del documento.
   - *menor:* nuevas secciones o contenido sustancial.
   - *parche:* correcciones o actualizaciones menores.
4. **Registrar la entrada en el changelog** (sección 13) y actualizar `fecha` y `commit_base` en el front matter.
5. **Regenerar el PDF:** `npm --prefix docs run generar`.
6. **Verificar** el PDF generado en `docs/pdf/`.

14.3 Fuente de verdad

Información	Fuente de verdad
Contenido del documento	`docs/DOCUMENTACION.md`
Versión del documento	Front matter de `DOCUMENTACION.md`
Dependencias de la app	`amazon-clone/package.json`
Configuración Firebase	`amazon-clone/src/firebase.js`
Estado del código	Repositorio y su historial de Git